Assignment 2: Technical Design

September 2026

1 Project Identification

Team Name	Big O(oof)
Team Members	Gayatri Taneja, Shivanshi Singh
Roll Numbers	1020241849, 1020241319
Assigned Project	Interactive Delhi Metro Route Planner and Visualiser
Repository Link	`https://github.com/Ashoka-CS-3810-Design-Practices-in-CS/project-1-BigOof`

Problem Statement The Interactive Delhi Metro Route Planner and Visualiser models the Delhi Metro network as a weighted, labelled graph in order to compute routes between two stations. The system supports two optimisation criteria: minimum travel time and minimum number of interchanges. The resulting route is presented through its station sequence, metro lines, interchange information, estimated travel time, and an interactive network visualisation.

Motivation The project applies graph-based route-finding algorithms to a practical transportation network while providing an interactive visualisation of the computed route. It provides a concrete application of shortest-path algorithms and graph data structures while separating the route computation from its presentation.

Intended Users The intended users are Delhi Metro passengers planning journeys between two stations, including regular commuters and visitors. The interface is designed so that users can obtain route information without requiring knowledge of the underlying graph representation or route-finding algorithms.

Main Goal The main goal of the application is to allow a user to select a source and destination station and obtain an optimal route according to the selected criterion: minimum travel time or minimum number of interchanges. The system then displays the corresponding station sequence, metro lines, interchange locations and count, estimated travel time, and highlighted route on the interactive map.

2 Algorithms

The routing engine implements two Dijkstra-based algorithms corresponding to the two optimisation criteria supported by the application. Standard Dijkstra's algorithm is used to minimise total travel time, while a state-augmented version of Dijkstra's algorithm is used to minimise the number of interchanges, with travel time used as a tie-breaker.

2.1 Minimum-Travel-Time Route

Purpose The minimum-travel-time algorithm uses standard Dijkstra's algorithm to compute a route from a source station to a destination station that minimises total estimated travel time, with travel time as the edge weight.

Input and Output

- **Input:** The metro graph $G = (V, E)$, a source station s, and a destination station d.
- **Output:** The minimum-travel-time route from s to d, together with predecessor information used for route reconstruction.

For an edge (u, v) with travel-time weight $w(u, v)$, the relaxation operation is:

$$d(v) = \min\left(d(v),\, d(u) + w(u, v)\right).$$

Algorithm

Algorithm 1 Minimum-Travel-Time Route

Require: Graph $G = (V, E)$, source s, destination d
Ensure: Minimum-travel-time route from s to d

```
for all v ∈ V do
    dist[v] ← ∞
    prev[v] ← NIL
end for
dist[s] ← 0
Q ← empty min-priority queue
Insert (0, s) into Q
while Q is not empty do
    (du, u) ← ExtractMin(Q)
    if du ≠ dist[u] then
        continue
    end if
    if u = d then
        break
    end if
    for all (u, v, w, c) ∈ Adj[u] do
        if dist[u] + w < dist[v] then
            dist[v] ← dist[u] + w
            prev[v] ← (u, c)
            Insert (dist[v], v) into Q
        end if
    end for
end while
Reconstruct route from d using prev
return reconstructed route =0
```

Correctness Intuition Dijkstra's algorithm maintains the smallest currently known travel time from the source to every discovered station. Because all edge weights are non-negative, once a station is extracted from the priority queue with its current minimum distance, no later path can provide a smaller distance to that station. Therefore, when the destination is finalised, its distance is the minimum possible total travel time from the source. **Complexity**

Using an adjacency-list representation and a binary-heap priority queue, the time complexity is

$$O\left((|V| + |E|) \log |V|\right).$$

The space complexity is $O(|V| + |E|)$, including the graph, distance and predecessor arrays, and priority queue. The standard binary-heap implementation has $O((|V|+|E|) \log |V|)$ time complexity and $O(|V| + |E|)$ space complexity.

Algorithm Selection Dijkstra's algorithm was selected because the minimum-travel-time objective is a single-source shortest-path problem with non-negative edge weights. A breadth-first search would not be appropriate because it minimises the number of edges rather than the total travel time.

2.2 Minimum-Interchange Route

Purpose The minimum-interchange algorithm computes a route that minimises the number of metro-line changes, using travel time as a tie-breaker among routes with the same interchange count. A scalar edge weight on the original station graph is insufficient because the interchange cost depends on the line used to reach the current station, so the routing state is augmented with the arrival line.

State Representation An augmented state is represented as

$$(u, \ell),$$

where u is the current station and ℓ is the metro line used to arrive at that station. At the source station, $\ell = \perp$ indicates that no metro line has yet been selected.

For an edge (u, v) belonging to line c, the transition is

$$(u, \ell) \to (v, c).$$

The interchange cost δ for this transition is

$$\delta = \begin{cases} 0, & \ell = c \text{ or } \ell = \perp, \\ 1, & \ell \neq c. \end{cases}$$

Each transition therefore has a cost pair (δ, w), where w is the estimated travel time. The cost pairs are compared lexicographically:

$$(a_1, b_1) < (a_2, b_2) \iff a_1 < a_2 \ \vee \ (a_1 = a_2 \wedge b_1 < b_2).$$

Thus, interchanges are minimised first, with travel time used to break ties.

Algorithm

Algorithm 2 Minimum-Interchange Route

Require: Graph $G = (V, E)$, source s, destination d
Ensure: Route from s to d minimising interchanges and then travel time

```
for all (v, ℓ) do
    cost[v, ℓ] ← (∞, ∞)
    prev[v, ℓ] ← NIL
end for
cost[s, ⊥] ← (0, 0)
Q ← empty min-priority queue
Insert (0, 0, s, ⊥) into Q
while Q is not empty do
    (c, u, ℓ) ← ExtractMin(Q)
    if c ≠ cost[u, ℓ] then
        continue
    end if
    for all (u, v, w, c_line) ∈ Adj[u] do
        if ℓ = ⊥ or ℓ = c_line then
            δ ← 0
        else
            δ ← 1
        end if
        newCost ← cost[u, ℓ] + (δ, w)
        if newCost < cost[v, c_line] then
            cost[v, c_line] ← newCost
            prev[v, c_line] ← (u, ℓ)
            Insert (newCost, v, c_line) into Q
        end if
    end for
end while
Select the minimum-cost state (d, ℓ) at destination d
Reconstruct route using prev
return reconstructed route =0
```

Correctness Intuition

The augmented state records both the current station and the line used to reach it, allowing the algorithm to distinguish between continuing on the current line and changing to a different line at the same station. Each transition adds zero to the interchange count when the same line is continued and one when the line changes, while the augmented state represents each possible arrival-line choice. Dijkstra's algorithm therefore finds the lexicographically minimum pair of interchange count and travel time.

Complexity

Let L denote the maximum number of metro lines associated with a station. The augmented graph contains at most $O(|V|L)$ states and $O(|E|L)$ transitions. Using a binary-heap priority queue, the time complexity is

$$O\left(L(|V| + |E|)\log(L|V|)\right).$$

The space complexity is

$$O(|V|L + |E|L),$$

including the augmented state representation, predecessor information, costs, and priority queue.

Algorithm Selection

State-augmented Dijkstra was selected because the interchange cost is dependent on the line used to reach a station. Standard Dijkstra on the original station graph cannot represent this dependency using only a station-level state.

2.3 Route Reconstruction and Interchange Detection

Purpose

After either route-finding algorithm has determined the optimal destination state, the route-processing component reconstructs the corresponding station sequence and determines the metro lines and interchange stations associated with the route.

Algorithm The predecessor information is followed backwards from the destination until the source is reached. The resulting sequence is then reversed to obtain

$$S_0 \to S_1 \to S_2 \to \cdots \to S_n,$$

where S_0 is the source and S_n is the destination. The line associated with each consecutive station pair is then examined. Whenever the line used for the current segment differs from the line used for the preceding segment, the shared station between the two segments is marked as an interchange. The resulting route contains the station sequence, line assignments, interchange information, and total travel time.

Complexity For a route containing k edges, reconstructing the predecessor chain and examining the route segments requires $O(k)$ time and $O(k)$ additional space for the reconstructed route. Since $k \leq |V| - 1$ for a simple route, this is $O(|V|)$ in the worst case.

Algorithm Selection Predecessor-based reconstruction was selected because both route-finding algorithms already maintain predecessor information while computing the optimal route.

Algorithm	**Objective**	**Time Complexity**	**Space Complexity**	**Reason for Selection**
Dijkstra	Minimum travel time	$O((\|V\| + \|E\|) \log \|V\|)$	$O(\|V\| + \|E\|)$	Suitable for single-source shortest paths with non-negative edge weights.
State-Augmented Dijkstra	Minimum interchanges, with travel time as a tie-breaker	$O(L(\|V\| + \|E\|) \log(L\|V\|))$	$O(L(\|V\| + \|E\|))$	Represents the arrival line, allowing interchange costs to depend on the current line.
Predecessor Reconstruction	Recover the selected route and interchange information	$O(\|V\|)$	$O(\|V\|)$	Uses predecessor information already maintained during route computation.

Table 1: Summary of route-finding and route reconstruction algorithms.

3 Data Structures

The application uses data structures that support efficient representation of the metro network, shortest-path computation, and route reconstruction. The principal structures are the adjacency list, priority queue, station and edge records, and augmented routing states.

3.1 Adjacency List

Stored Information The metro network is represented using an adjacency list. For each station, the structure stores its directly connected neighbouring stations together with the metro line and estimated travel time associated with each connection. Conceptually, the representation is

$$\text{station_id} \longrightarrow [(\text{neighbour}, \text{line}, \text{travel time})].$$

Suitability An adjacency list is suitable because the metro graph is sparse: each station has relatively few direct connections. It avoids storing entries for unconnected station pairs and allows the routing algorithms to iterate directly over the neighbours of the current station.

Important Operations The main operations are retrieving a station's adjacency list and iterating over its connections. Accessing the list takes $O(1)$ time, while traversing all neighbours of station u takes $O(\deg(u))$ time.

Space Complexity The adjacency list requires $O(|V| + |E|)$ space.

Interaction with Other Components The data layer constructs the adjacency list from the validated metro dataset. The routing engine receives this graph and uses the adjacency lists during Dijkstra-based route computation.

3.2 Priority Queue

Stored Information A binary-heap min-priority queue stores unsettled routing states with their current best-known costs. For minimum travel time, an entry contains a station and travel-time estimate; for minimum interchanges, it additionally contains the arrival line and lexicographic cost.

Suitability A min-priority queue is required by Dijkstra's algorithm to efficiently select the unsettled state with the smallest current cost. A binary heap provides efficient insertion and minimum-element extraction.

Important Operations The main operations are inserting a routing state and extracting the minimum-cost state. For a binary heap, both operations take $O(\log |Q|)$ time, while accessing the minimum element takes $O(1)$ time.

Space Complexity The priority queue requires $O(|Q|)$ space.

Interaction with Other Components The priority queue is used internally by the routing engine. The graph provides neighbouring states to the routing algorithm, which inserts improved states into the queue and repeatedly extracts the minimum-cost state. The queue is not exposed to the API or frontend.

3.3 Station, Line, and Edge Records

Stored Information

The persistent metro consists of:

- **Station:** station identifier, station name, and coordinates;

- **Line:** line identifier, line name, and display colour;
- **Edge:** source station, destination station, metro line identifier, and estimated travel time.

These records provide the source data from which the in-memory graph is constructed.

Suitability Separating stations, lines, and connections avoids duplicating station and line information across the network dataset.

Important Operations The records support station and line lookup, connection loading, and validation of station and line references. The precise complexity of database lookups depends on the indexing strategy. These operations are performed during data loading and validation rather than during core shortest-path computation.

Interaction with Other Components The Data Layer loads and validates these records before constructing the in-memory adjacency list. The Routing Engine then operates on the resulting graph rather than querying the persistent data store during route requests.

3.4 Augmented Routing State

Stored Information An augmented routing state stores the current station, arrival line, lexicographic cost (interchanges, travel time), and predecessor state.

Suitability The augmented state is necessary because interchange cost depends on the line used to reach the current station; a station-only state cannot represent this dependency.

Important Operations The main operations are creating a state, updating its cost, and storing its predecessor. Determining the cost of a transition takes $O(1)$ time once the current state and edge information are available.

Space Complexity

If L is the maximum number of lines associated with a station, there are at most

$$O(|V|L)$$

augmented states.

Interaction with Other Components The Routing Engine creates augmented states from the metro graph and uses their predecessor information for route reconstruction.

3.5 Data Structure Summary

Data Structure	Stored Information	Important Operations	Justification
Adjacency List	Neighbouring stations, metro lines, and travel times	Neighbour lookup, neighbour traversal	Efficient representation for the sparse metro network
Binary-Heap Priority Queue	Routing states and their current costs	Insert, Extract-Min	Efficient minimum-cost selection for Dijkstra's algorithm
Station/Line/Edge Records	Station, line, connection, coordinate, and travel-time data	Lookup, validation, data loading	Structured persistent representation of the metro network
Augmented Routing State	Current station, arrival line, lexicographic cost, predecessor	State transition, cost update, predecessor storage	Represents line-dependent interchange costs

Table 2: Principal data structures used by the system

4 Software Architecture and Design

The system is organised into four components: the Presentation Layer, Application/API Layer, Routing Engine, and Data Layer.

4.1 Architecture Overview

A route request flows from the browser through the API and routing engine, with the resulting route returned to the frontend for visualisation.

4.2 Data Layer

The Data Layer provides the persistent source of structured metro network information. The metro data is stored in SQL tables for stations, lines, and connections. The validated data is then used to construct the in-memory metro graph represented by an adjacency list.

The Data Layer therefore separates persistent data storage and validation from the algorithms that operate on the resulting graph.

4.3 Routing Engine

The Routing Engine contains the in-memory metro graph and executes the selected route-finding algorithm. It uses Dijkstra's algorithm for minimum-travel-time routes and state-augmented Dijkstra for minimum-interchange routes. The engine returns the computed route to the Application/API Layer.

4.4 Application and API Layer

The Application/API Layer provides the interface between the frontend and the Routing Engine. The backend is implemented using Flask and exposes a route endpoint through which the frontend submits routing requests.

A route request has the following logical structure:

```
POST /route
{
    "source": <station>,
    "dest": <station>,
    "mode": <routing_mode>
}
```

The Application/API Layer performs the following operations:

1. Validates the source station, destination station, and requested routing mode.
2. Selects the appropriate route-finding algorithm based on the requested mode.
3. Passes the validated request to the Routing Engine.
4. Processes the returned route to determine the station sequence, journey duration, and interchange information.
5. Returns the processed result to the frontend in JSON format.

The API therefore isolates the frontend from the implementation details of the routing algorithms and graph representation.

4.5 Presentation Layer

The Presentation Layer provides the interactive browser-based interface. It is implemented using HTML, JavaScript, Tailwind CSS, and the Transit-Map visualisation component.

The interface allows the user to search for and select source and destination stations, choose the routing criterion, and submit a route request. After receiving the API response, the frontend displays the route summary and overlays or highlights the computed route on the metro map.

The Presentation Layer also handles loading and error states so that API requests and invalid inputs are communicated to the user without exposing the internal routing implementation.

4.6 Architecture Diagram

Figure 1 shows the main components and their interactions.

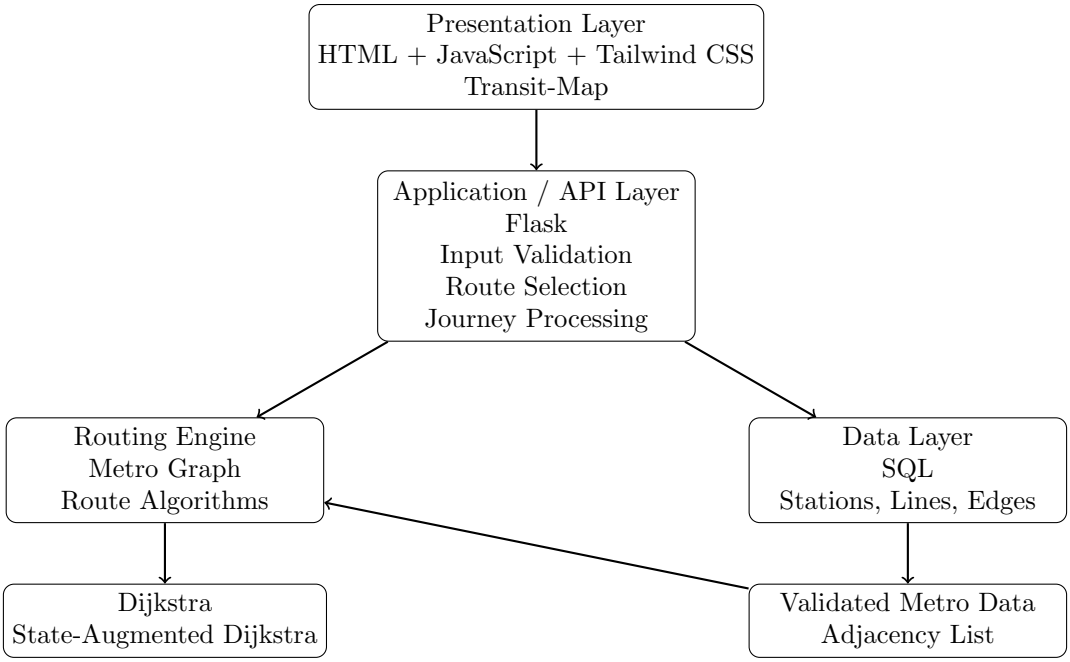


Figure 1: Software architecture of the Delhi Metro Route Planner.

5 Technology Stack

The system uses the following technologies for its presentation, application logic, API, visualisation, and persistent data storage.

Category	**Technology**	**Reason for Selection**
Frontend	HTML	Provides the standard structure required for the browser-based interface.
Frontend	JavaScript	Supports interactive user input and communication with the backend API.
Styling	Tailwind CSS	Provides a consistent and responsive styling framework for the interface.
Visualisation	Transit-Map	Provides the metro network visualisation required to display and highlight computed routes.
Backend	Python	Supports implementation of the routing algorithms and application logic.
API	Flask	Provides a lightweight interface between the browser frontend and the Python backend.
Data Storage	SQL	Provides structured persistent storage for station, line, and connection data.
Data Exchange	JSON	Provides a structured format for exchanging routing requests and results between the frontend and API.
Version Control	Git / GitHub	Provides version control and repository-based collaboration for the project.

Table 3: Technology stack and rationale for selection.